Capybara Road: documentação do desenvolvimento

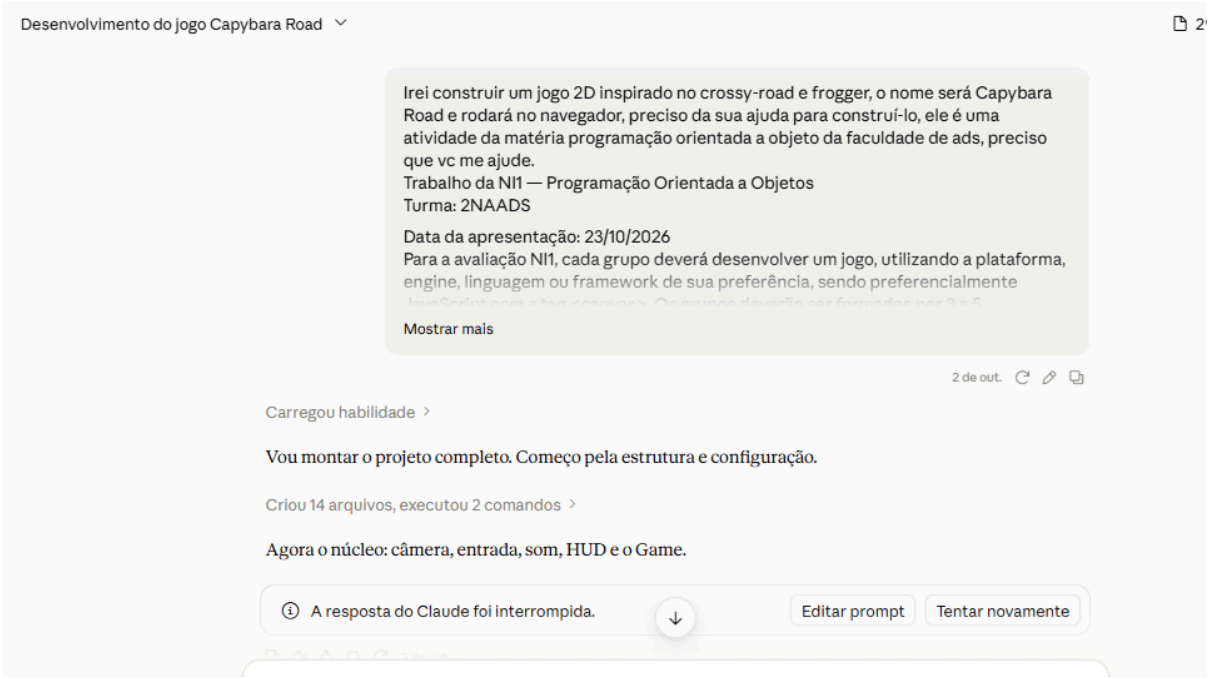
Registro de como o Capybara Road foi feito para a NI1 de POO (ADS, 2NAADS): as ferramentas de IA usadas e a comparação entre elas, os prompts, os bugs encontrados e corrigidos, com código antes e depois, e as evidências.

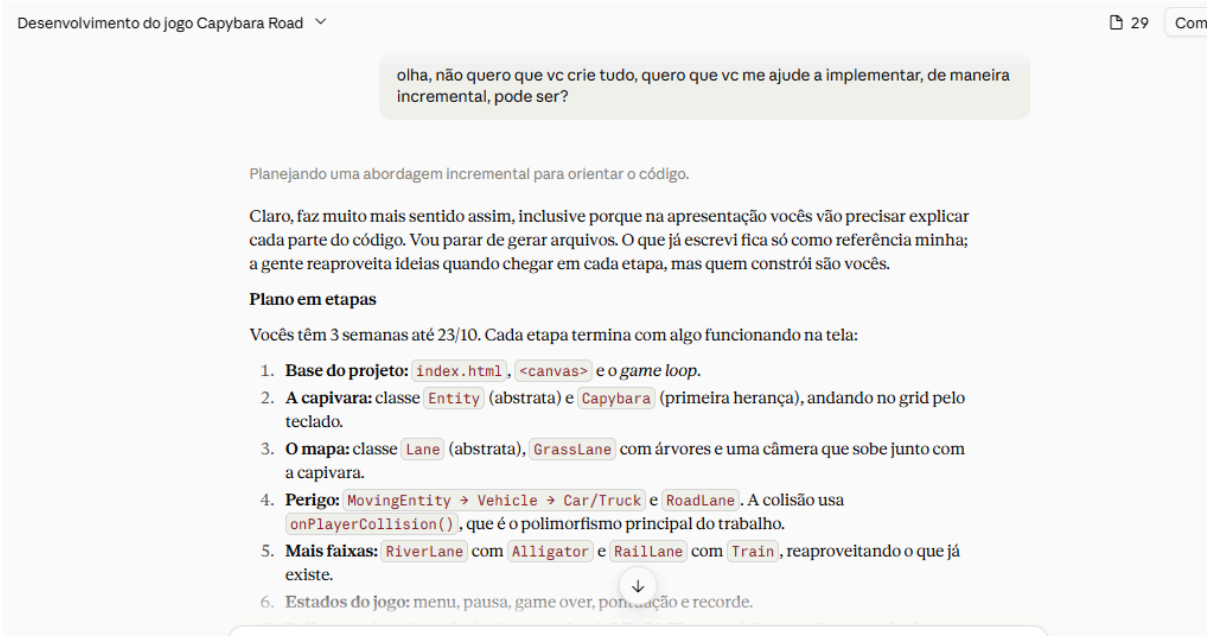
Ferramentas de IA e comparação

Claude (Anthropic)

- **Para que usamos:** como assistente de programação. Relembrou os conceitos de POO, propôs a estrutura de classes, gerou o código de cada etapa, ajudou a achar e corrigir bugs e montou o README.
- **Em quais etapas:** em todo o código, de 04/10 a 13/10 (game loop, capivara, mapa, veículos, rio, trilho, onça, pontuação, telas e sons), e depois na documentação e na apresentação.
- **Exemplo de prompt:** "Quero que a onça fique mais rápida e a capivara fique mais lenta na água"
- **O que voltou:** uma constante nova (`SWIM_HOP_DURATION`) no `config.js`, a mudança no `updateHop()` da `Capybara` para o pulo demorar mais na água e um aviso de que a onça mais rápida deixaria as medalhas mais fáceis. O código veio testado com o teste automático.
- **Pontos fortes:** explica cada conceito com calma, trabalhou do jeito incremental que o grupo pediu (um passo e um commit por vez), testava o código antes de entregar e respeitou as decisões do grupo, como seguir a bíblia visual.
- **Pontos fracos:** não consegue jogar nem ouvir o jogo, então algumas soluções precisaram de ajuste depois que o grupo testou: o controle no nado levou três tentativas e o som do pulo precisou de duas rodadas. No bug do clique, primeiro apontou a causa errada. As respostas às vezes são longas.

- **Print:**





ChatGPT (OpenAI)

- **Para que usamos:** para quatro coisas. (1) No começo, para organizar o trabalho: entender o enunciado, dividir o projeto em etapas e definir a ideia do jogo, uma capivara atravessando a rua com identidade brasileira. (2) Para criar a identidade visual. (3) Para escrever prompts melhores para o Claude. (4) Para a comparação de código com o Claude.

- **Em quais etapas:** no planejamento inicial, antes do código; ao longo do projeto, para montar prompts; no design do menu (11/10); e na comparação final (21/10).
- **Exemplo de prompt:** Prompt da identidade visual (resumido):

  CAPYBARA ROAD — Brand Identity & Logo Design

  Create a distinctive, playful and memorable visual identity for CAPYBARA ROAD, a casual arcade game inspired by the classic road-crossing gameplay genre. The main character is a cute Brazilian capybara navigating chaotic streets, highways, rivers and environments inspired by everyday life in Brazil.

  Core Concept: fun, chaotic, charming, humorous and unmistakably Brazilian. (...)

  Logo Design: bold, chunky, rounded custom lettering; a stylized capybara mascot integrated into the lettering; subtle road elements. (...)

  Brazilian Visual Language: tropical green, sunny yellow, warm orange, sky blue, earthy brown and dark asphalt gray. (...)

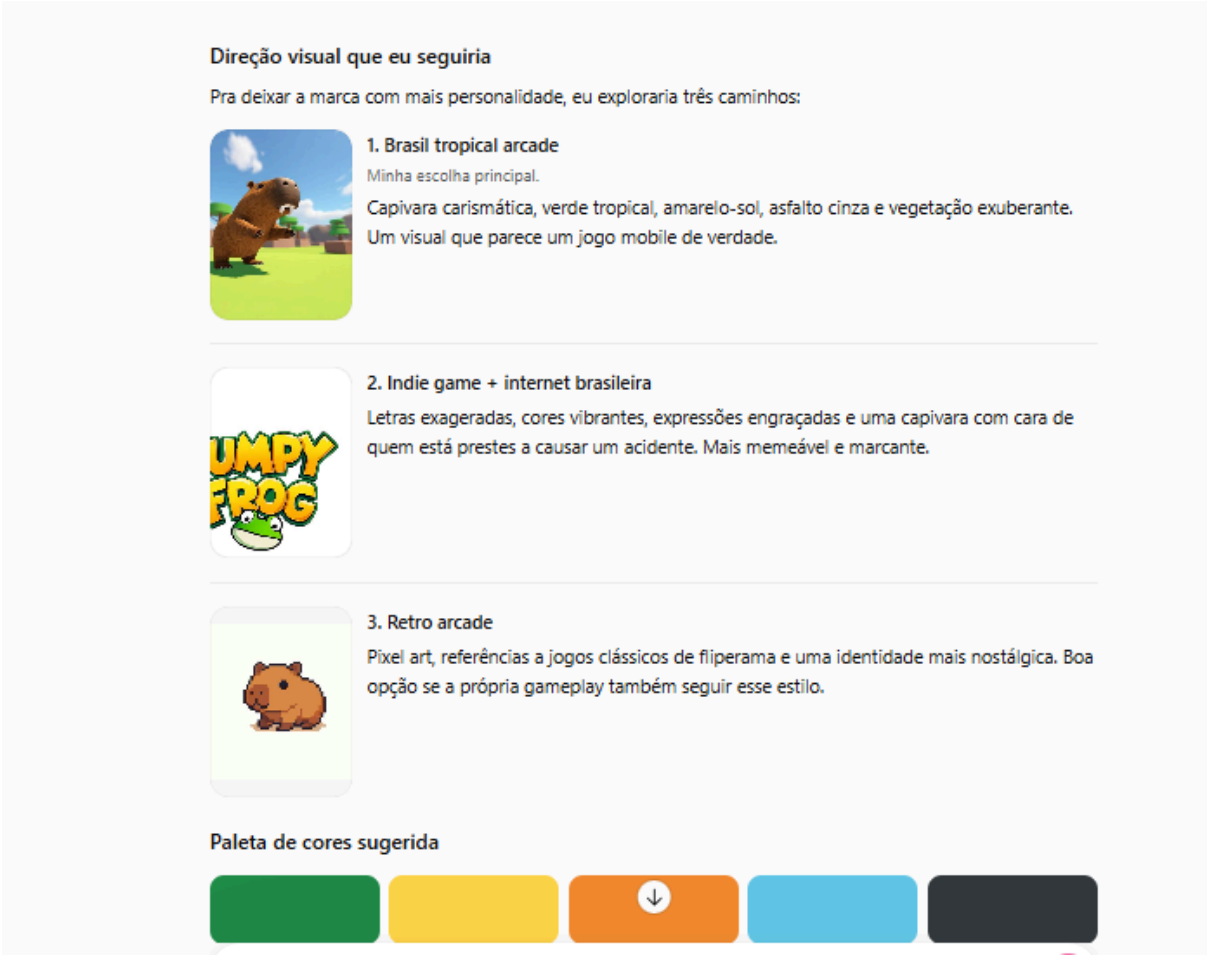
  Art Direction: stylized 3D-inspired cartoon, low-poly characters, bold silhouettes, vibrant colors. (...)

  Deliverables: primary logo, standalone mascot, app icon, color palette with HEX codes, typography direction, background pattern.

  The title must be spelled exactly: "CAPYBARA ROAD".

- **O que voltou:** três direções visuais, a paleta de cores e a arte em pixel art do logo, do botão e das nuvens. O grupo ficou com o estilo pixel art (a direção "Retro arcade") para o menu, e a paleta sugerida é muito parecida com a que o jogo usa. No teste de comparação, veio a classe abstrata `Entity` com `new.target`, as subclasses `Car`, `Orange` e `Tree`, uma função `checkCollisions()` sem teste de tipo e uma explicação dividida em seções.
- **Pontos fortes:** gratuito; ótimo para organizar ideias e para a parte visual; no código, veio correto e curto, com uma explicação bem organizada e didática, e avisou que a árvore não bloqueia o jogador.
- **Pontos fracos:** no código, entregou só trechos para encaixar, sem uma demo para testar; a laranja não sai da lista, só fica marcada como coletada; os blocos de código vieram sem a linguagem marcada.

- **Print:**



Comparação

Prompt usado nas duas: "Estou fazendo um jogo estilo Crossy Road em JavaScript puro com canvas, para um trabalho de Programação Orientada a Objetos. Crie uma classe abstrata Entity (com x, y, width, height e os métodos draw e onPlayerCollision) e três subclasses: Car, que dá game over quando o jogador encosta; Orange, que dá +50 pontos e some; e Tree, que não faz nada. Use herança e polimorfismo, de forma que o código que verifica a colisão não precise de nenhum if para saber o tipo da entidade. Comente o código em português e explique onde está o polimorfismo."

Observação: o Claude foi usado no plano pago e o ChatGPT no gratuito.

Funcionamento: o código das duas funcionou de primeira. O Claude entregou uma demo jogável (index.html e entities.js), em que se move com as setas e reinicia com R. O ChatGPT entregou trechos de código, e o grupo teria que montar o jogo em volta deles para testar.

Classe abstrata: as duas usaram new.target no construtor da Entity para impedir que ela seja criada direto. Nas duas, os métodos draw e onPlayerCollision lançam erro se a subclasse não os sobrescrever.

Herança: as duas usaram extends Entity e super(). No Claude, as subclasses herdam o método collidesWith e um update padrão. No ChatGPT, herdam o método intersects.

Polimorfismo: nas duas, ele aparece na colisão, com a chamada entity.onPlayerCollision() sem nenhum teste de tipo, e também no draw(). No Claude, aparece ainda no update(), em que só o carro se move.

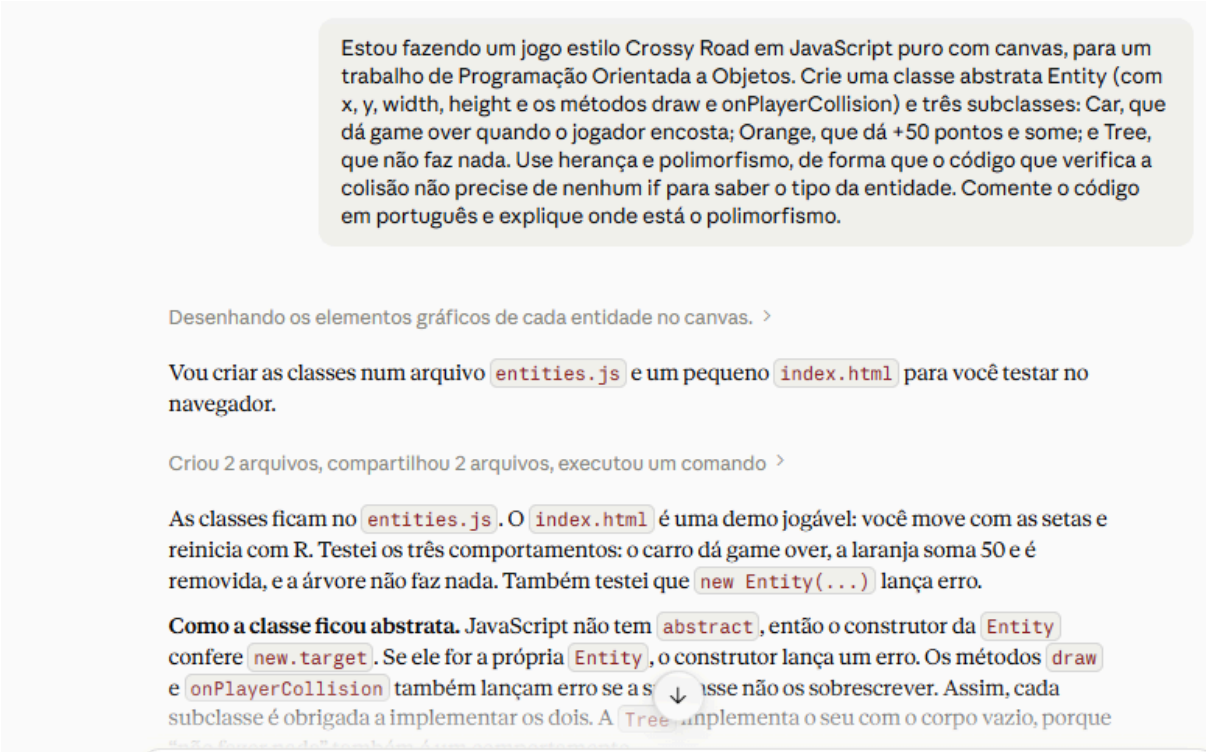
Laranja: no Claude, a laranja é marcada como removida e o jogo a tira da lista. No ChatGPT, ela é marcada como coletada e para de ser desenhada, mas continua na lista.

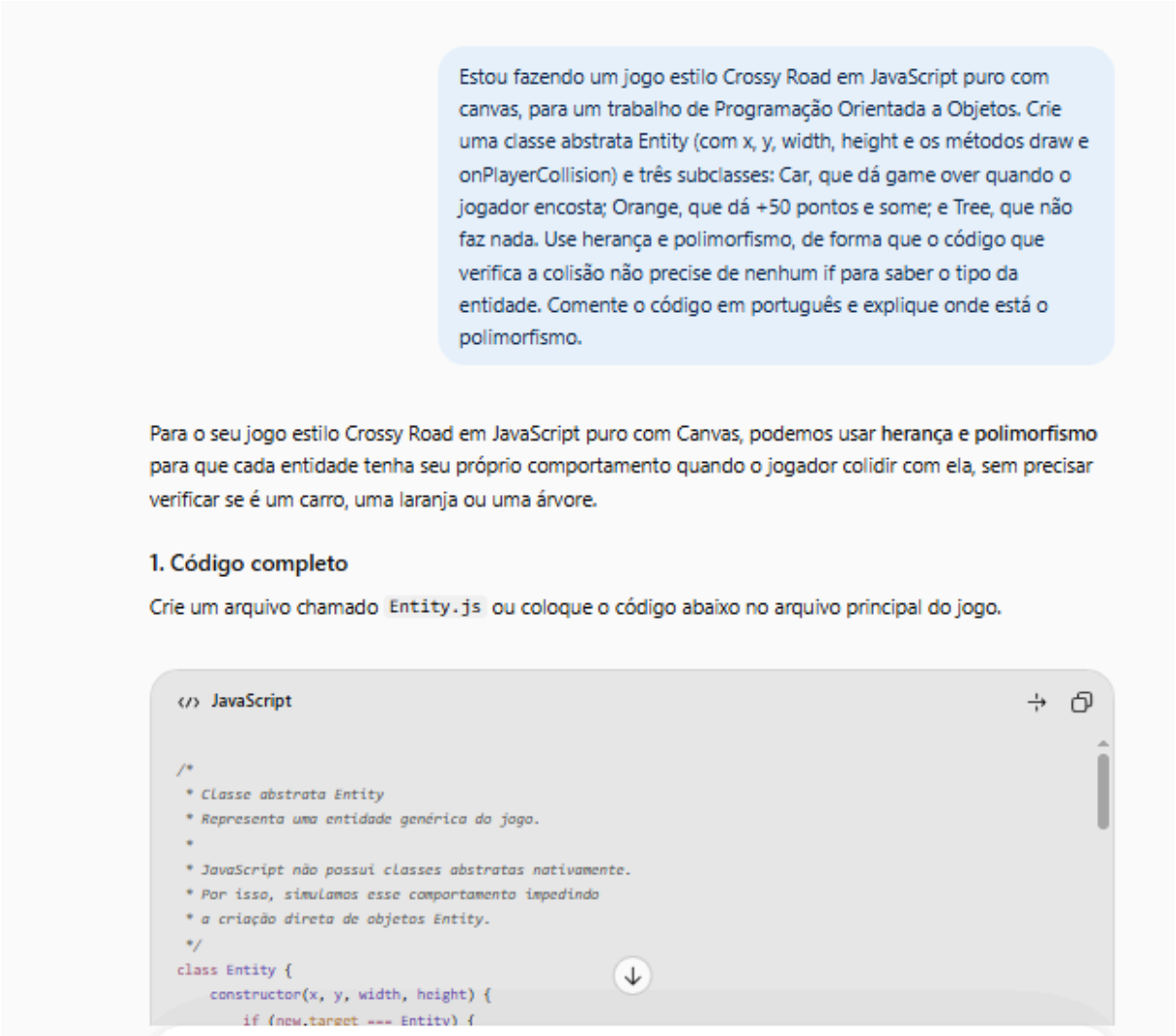
Explicação: a do Claude foi mais curta e técnica e citou o despacho dinâmico e o princípio aberto/fechado. A do ChatGPT foi mais longa e didática, dividida em seções, com um resumo pronto para a apresentação.

Erros: não encontramos erros em nenhuma das duas.

Tempo: as duas chegaram a uma solução boa na primeira resposta. O Claude sendo mais completo mas com a ressalva de ser pago.

Print:





Conclusão: as duas IAs acertaram os conceitos que a NI1 pede: classe abstrata com new.target, herança com extends e polimorfismo sem nenhum teste de tipo. A diferença está no que cada uma entregou. O Claude foi além do pedido, com uma demo pronta para rodar e a laranja removida de verdade. O ChatGPT deu uma explicação mais didática e organizada, boa para estudar, e de graça. A comparação não é totalmente justa, porque usamos o plano pago do Claude e o gratuito do ChatGPT. Por isso o grupo usou cada uma no que ela fazia melhor: o ChatGPT para organizar o trabalho, escrever prompts e criar a arte do menu, e o Claude para construir o código.

Bugs e correções

Bug 1: o clique dos botões não fazia som

Sintoma: depois de adicionar os sons, nenhum botão fazia barulho ao ser clicado.

Causa: o som tocava antes da ação do botão. Mas quase toda ação (jogar, menu, pause) chama `sound.stopAll()`, que cortava o clique no mesmo instante. A primeira suspeita foi o arquivo de som ser baixo demais; trocar o arquivo não resolveu.

Antes (`js/screens/Screen.js`):

```
if (button.contains(x, y)) {
    this.game.sound.play('click');
    button.click(); // pode chamar stopAll() e cortar o clique
    return;
}
```

Depois:

```
if (button.contains(x, y)) {
    button.click();
    // o som vem DEPOIS: a ação do botão pode chamar stopAll()
    this.game.sound.play('click');
    return;
}
```

Bug 2: a capivara "desobedecia" no rio

Sintoma: depois de deixar o nado mais lento, quem apertava as teclas rápido perdia o controle. No teste de fazer um quadrado (W A S D), a capivara pulava movimentos.

Causa: a fila de teclas guardava só 2 movimentos. Com a braçada mais lenta, as teclas chegavam mais rápido do que eram executadas, e as extras eram descartadas. Uma simulação de 8 teclas mostrou que só 5 eram executadas.

Tentativa que piorou: guardar só a última tecla (`consumeLatestMove`). No quadrado WASD, as teclas do meio sumiam.

Antes (`js/core/InputHandler.js`):

```
static MAX_QUEUE = 2;
```

Depois (commit `c073a00`):

```
static MAX_QUEUE = 5;
```

Bug 3: a capivara sumia da tela

Sintoma: ao apertar uma seta, a capivara desaparecia.

Causa: o código do pulo usava `CONFIG.HOP_DURATION` e `CONFIG.HOP_HEIGHT`, que ainda não existiam no `config.js`. Uma conta com `undefined` dá `NaN`, e a capivara era desenhada numa posição inválida.

Antes (`js/entities/Capybara.js`, com o `config.js` sem as constantes):

```
const t = Math.min(this.hopTimer / CONFIG.HOP_DURATION, 1); // undefined → NaN

this.x = Utils.lerp(this.startX, this.targetX, t);        // x vira NaN
```

Depois (`js/config.js`):

```
HOP_DURATION: 0.12, // duração de um pulo, em segundos

HOP_HEIGHT: 10,     // altura do arco do pulo, em pixels
```

Bug 4: a onça passava por cima das árvores

Sintoma: a onça saía da floresta atravessando as árvores desenhadas.

Causa: a floresta era só um desenho de fundo com árvores em todas as colunas, sem nenhum caminho.

Antes (`js/lanes/SceneryLane.js`):

```
class SceneryLane extends Lane {
    isBlocked(col) { return true; }
    drawBackground(ctx, camera) {
        // uma copa de árvore desenhada em cada coluna
    }
}
```

Depois (commit `d403d83`):

```
class SceneryLane extends GrassLane {
    plantForest() {
        for (let col = 0; col < CONFIG.COLS; col++) {
            this.blockedCols.add(col);
            if (col === this.trailCol) continue; // trilha da onça
            // árvore ou pedra nas outras colunas
```

```
    }
  }
}
```

A onça também ganhou um salto ao passar pela cerca (`Jaguar.getFenceLeap()`).